¿Qué es la refactorización del código?

La refactorización del código es una práctica de desarrollo de software que altera la estructura interna del código sin modificar su comportamiento externo ni afectar a sus funciones. Estos pequeños cambios pretenden hacer el código más legible y fácil de mantener.

Martin Fowler popularizó la práctica a través de su libro Refactoring, que se publicó por primera vez en 1999. La refactorización de código puede ayudar a eliminar los "olores de código ", que Fowler define como "una indicación superficial que generalmente corresponde a un problema más profundo en el sistema". Agrega que los olores de código son "rápidos de detectar o olfatear" y cita métodos largos y clases que contienen solo datos y ningún comportamiento como instancias de olores de código.

Algunos ejemplos de refactorización de código incluyen corregir formatos incorrectos, cambiar el nombre de variables no descriptivas, eliminar funciones duplicadas o no empleadas y dividir métodos grandes y largos en bloques más pequeños y manejables. Estas modificaciones triviales que preservan el comportamiento tienen menos probabilidades de romper código o introducir errores, pero su efecto incremental puede ayudar a optimizar el rendimiento.

Técnicas de refactorización de código

La refactorización puede parecer una tarea sencilla, pero algunas tácticas pueden ayudar a los desarrolladores de software a adoptar un enfoque más estratégico:

- **Abstracción**
- **Composición**
- **Mover características**
- **Refactorización rojo-verde**
- **Simplificación**

Beneficios de la refactorización de código

Piense en la refactorización de código como ordenar una habitación todos los días para que la limpieza al final de la semana sea mucho más fácil y rápida. El objetivo de la refactorización de código es reducir la deuda técnica, que se acumula como resultado de que los programadores tomen atajos, como duplicar la lógica, no seguir los estándares de programación o usar nombres de variables poco claros.

A continuación, se presentan algunos beneficios que los equipos de desarrollo de software pueden obtener de la refactorización de código:

- **Disminución de la complejidad**
- **Mantenimiento mejorado**
- **Mejora de la legibilidad del código**
- **Mayor velocidad**

Cuándo NO Re factorizar

- En código heredado sin ninguna prueba.
- Cuando la funcionalidad está a punto de ser descontinuada.
- Si el plazo de entrega es extremadamente ajustado y el código no es crítico.
- Si el equipo no tiene experiencia en la técnica a aplicar.

Fragmento del código

Producto

```java
public class Producto {

    String id, nombre;
    double precio;
    int cantidadVendida = 0;

    public Producto(String id, String nombre, double precio) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
```

public class Producto {

String id, nombre;

double precio;

int cantidadVendida = 0;

public Producto(String id, String nombre, double precio) {

this.id = id;

this.nombre = nombre;

this.precio = precio; }

public String getId() {

return id; }

```
    public void setId(String id) {
        this.id = id;}
    public String getNombre() {
        return nombre;}
Public void setNombre(String nombre) {
        this.nombre = nombre;  }
    public double getPrecio() {
        return precio;
    }
    public void setPrecio(double precio) {
        this.precio = precio;
    }
    public int getCantidadVendida() {
        return cantidadVendida;
    }

    public void setCantidadVendida(int cantidadVendida) {
        this.cantidadVendida = cantidadVendida;
    }
}
```

Vendedor

```java
public class Vendedor {

    String tipoDoc, numDoc, nombres, apellidos;
    double ventasTotales = 0.0;

    public Vendedor(String tipoDoc, String numDoc, String nombres, String apellidos) {
        this.tipoDoc = tipoDoc;
        this.numDoc = numDoc;
        this.nombres = nombres;
        this.apellidos = apellidos;
    }

    public String getTipoDoc() {
        return tipoDoc;
    }

    public void setTipoDoc(String tipoDoc) {
        this.tipoDoc = tipoDoc;
    }

    public String getNumDoc() {
```


```java
public class Vendedor {

    String tipoDoc, numDoc, nombres, apellidos;
    double ventasTotales = 0.0;

    public Vendedor(String tipoDoc, String numDoc, String nombres, String apellidos) {
        this.tipoDoc = tipoDoc;
        this.numDoc = numDoc;
        this.nombres = nombres;
        this.apellidos = apellidos;
    }

    public String getTipoDoc() {
        return tipoDoc;
    }

    public void setTipoDoc(String tipoDoc) {
```

```
    this.tipoDoc = tipoDoc;
}

public String getNumDoc() {
    return numDoc;
}

public void setNumDoc(String numDoc) {
    this.numDoc = numDoc;
}

public String getNombres() {
    return nombres;
}

public void setNombres(String nombres) {
    this.nombres = nombres;
}

public String getApellidos() {
    return apellidos;
}

public void setApellidos(String apellidos) {
    this.apellidos = apellidos;
}
```

```
    public double getVentasTotales() {
        return ventasTotales;
    }

    public void setVentasTotales(double ventasTotales) {
        this.ventasTotales = ventasTotales;
    }


}
```